

GeoLog

UNIPE - Topicos Avancados em Banco de Dados

Professor: Me. Ricardo Roberto de Lima

Aluno: Davi Medeiros Dantas Soares

1. Visao Geral da Solucao

O GeoLog foi implementado em um unico arquivo Python executado com Streamlit. A solucao aplica persistencia poliglota: os dados cadastrais e relacionais de motoristas e veiculos ficam no SQLite, enquanto a telemetria de alta variacao - coordenadas, temperatura, velocidade e timestamp - fica no MongoDB. A aplicacao integra as duas fontes em memoria por meio do campo veiculo_id, apresentando mapa, indicadores e graficos em uma unica interface.

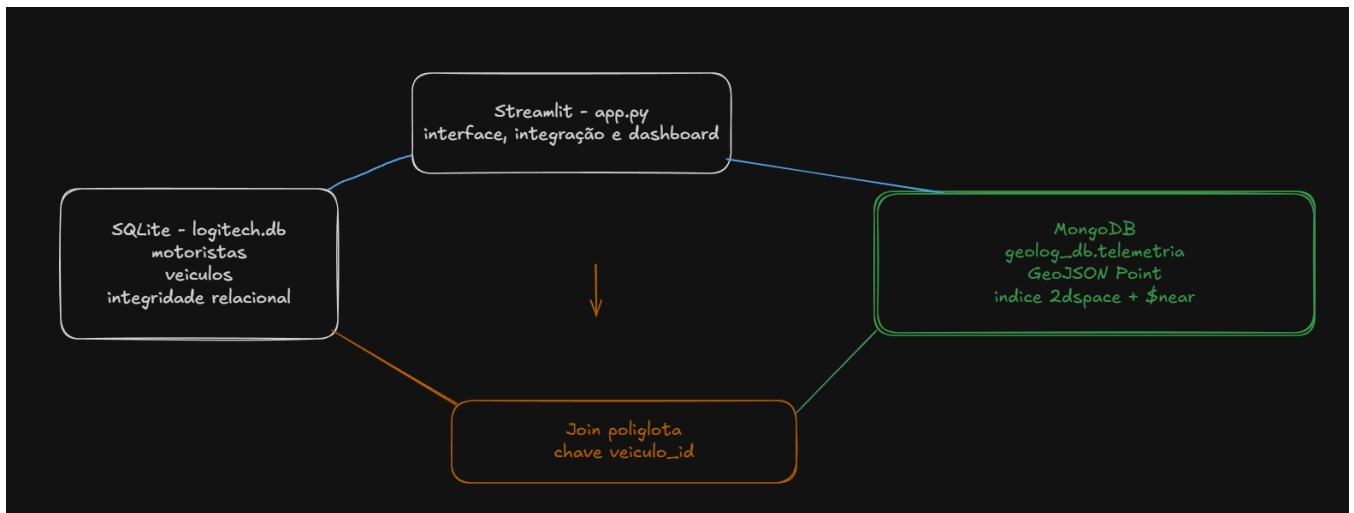


Figura 1 - Arquitetura poliglota implementada no GeoLog.

2. Persistencia e Inicializacao

SQLite. O arquivo logitech.db e criado automaticamente. A tabela motoristas armazena id, nome, CNH e status; a tabela veiculos armazena id, placa, modelo e motorista_id, mantendo a referencia ao motorista. O seed usa INSERT OR IGNORE, permitindo novas execucoes sem duplicar os registros iniciais.

MongoDB. A aplicacao conecta em mongodb://localhost:27017, considerando o banco executado em container Docker. A colecao geolog_db.telemetry recebe documentos com localizacao GeoJSON no formato Point, cuja ordem das coordenadas e [longitude, latitude]. Na inicializacao e criado via codigo o indice location_2dsphere, requisito necessario para consultas geoespaciais por proximidade.

3. Consultas Geoespaciais e Integracao

A busca por raio utiliza o operador \$near. O usuario escolhe um ponto de referencia e um raio em quilometros; antes da consulta, o raio e convertido para metros, unidade esperada por \$maxDistance. O resultado e exibido em um mapa Folium com marcador do ponto central, circulo representando o raio e marcadores dos veiculos retornados.

Trecho conceitual da consulta:

```
location: { "$near": { "$geometry": { "type": "Point", "coordinates": [longitude, latitude] }, "$maxDistance":  
raio_km * 1000 } }
```

Para obter apenas a telemetria mais recente de cada veiculo, o MongoDB executa um aggregation pipeline: primeiro ordena por veiculo e timestamp decrescente; depois agrupa por veiculo_id e seleciona o primeiro documento de cada grupo; por fim restaura o documento original. Essa etapa fornece um estado atual da frota para o dashboard.

O join poliglota nao ocorre dentro de nenhum dos bancos. O SQLite retorna os dados relacionais em um DataFrame e o MongoDB retorna a ultima telemetria de cada veiculo. A aplicacao converte os documentos em estrutura tabular e executa merge(..., on="veiculo_id", how="left"). Assim, motorista e placa permanecem sob responsabilidade do banco relacional, enquanto temperatura, velocidade e coordenadas permanecem no MongoDB.

4. Dashboard e Visualizacao

Componente	Implementacao
KPIs	Frotas ativas, temperatura media e alertas de velocidade acima de 80 km/h.
Mapa	Folium + MarkerCluster, ponto de referencia, raio e veiculos retornados por \$near.
Visao unificada	st.dataframe com motorista, placa, temperatura, velocidade, latitude e longitude.
Grafico 1	Plotly line chart com historico de temperatura por veiculo.
Grafico 2	Plotly pie/donut com distribuicao do status dos motoristas.

A interface e dividida em tres abas - Geoprocessamento, Visao Unificada e Dashboard - para evidenciar os requisitos do desafio. O uso de Streamlit centraliza a demonstracao sem necessidade de um frontend separado.

5. Bonus - Simulacao de Telemetria

O botao Simular Movimentacao implementa o desafio bonus. Para cada veiculo, a funcao recupera sua ultima telemetria e insere um novo documento no MongoDB com pequena variacao aleatoria de latitude e longitude, temperatura e velocidade. O novo timestamp e gerado em UTC. Em seguida, st.rerun() reexecuta a aplicacao, fazendo o mapa, os KPIs e os graficos refletirem os novos registros sem reiniciar manualmente o programa.

6. Fluxo de Execucao e Demonstracao

O fluxo de inicializacao e simples: primeiro o SQLite e criado e populado; em seguida a aplicacao testa a conexao com o MongoDB, cria os indices e insere o seed somente se a colecao estiver vazia. Depois disso, a interface calcula os dados unificados e renderiza o dashboard.

Etapa	Acao
1	Subir o MongoDB em Docker e expor a porta 27017 no host.
2	Executar o Streamlit com: streamlit run app.py
3	Selecionar um ponto de referencia e variar o raio da busca.
4	Confirmar no mapa quais veiculos foram retornados pela consulta \$near.
5	Abrir a visao unificada e observar o join entre dados SQLite e MongoDB.
6	Usar Simular Movimentacao para criar novas telemetrias e atualizar o dashboard.

7. Conclusao

A implementacao atende ao objetivo de separar responsabilidades conforme o tipo de dado. O SQLite e utilizado onde ha estrutura relacional e integridade entre entidades; o MongoDB e utilizado para telemetria semiestruturada e consultas geoespaciais. O indice 2dsphere, o operador \$near, o aggregation pipeline e o join realizado na camada de aplicacao demonstram os principais conceitos de persistencia poliglota solicitados. O dashboard unifica a apresentacao operacional e o simulador adiciona um fluxo simples de telemetria em tempo real para o desafio bonus.

Tecnologias empregadas: Python, Streamlit, sqlite3, SQLite, PyMongo, MongoDB em Docker, GeoJSON, Folium, streamlit-folium, Pandas e Plotly.

Observacao: este relatorio descreve somente comportamentos presentes no codigo entregue.